분산시스템에서의 검색 엔진 사용과 연동 시스템의 구현

분산시스템 중간시험 대체 과제

박종현, 공과대학 컴퓨터정보통신공학과
jonghyeon@jnu.ac.kr

과제 목표

[제공해야하는 결과물]

1. IndexCreate.jar → 10K.ID.CONTENTS를 이용해 ElasticSearch에 인덱스를 만드는 Hadoop 프로그램
2. IndexCreate.java → IndexCreate.jar 소스코드
3. 보고서 1단원: ElasticSearch 설치 매뉴얼 및 설치결과 스냅샷
4. 보고서 2단원: Hadoop + ElasticSearch 연동 프로그램 구동결과 : 인덱스 생성 + 검색 기능 검증
   검색기능검증은 다음 기능 포함 : [1]title검색, [2]body검색, [3]url 검색, [4]상태확인

1. Elasticsearch의 설치

Elasticsearch는 Docker(도커) 컨테이너에 의존성을 가지고 있어 Elasticsearch를 실행하는 환경에는 도커 환경이 동작하고 있어야 한다.

본 과제의 실행 환경은 Windows Subsystem for Linux 2 (WSL)이다. 도커는 WSL 시스템에서는 호스트인 윈도우에 Docker Desktop(도커 데스크톱)을 설치한 후 백엔드 엔진으로 WSL 시스템을 사용하는 방향을 권장하고 있다: https://docs.docker.com/desktop/features/wsl/

```
❯ docker

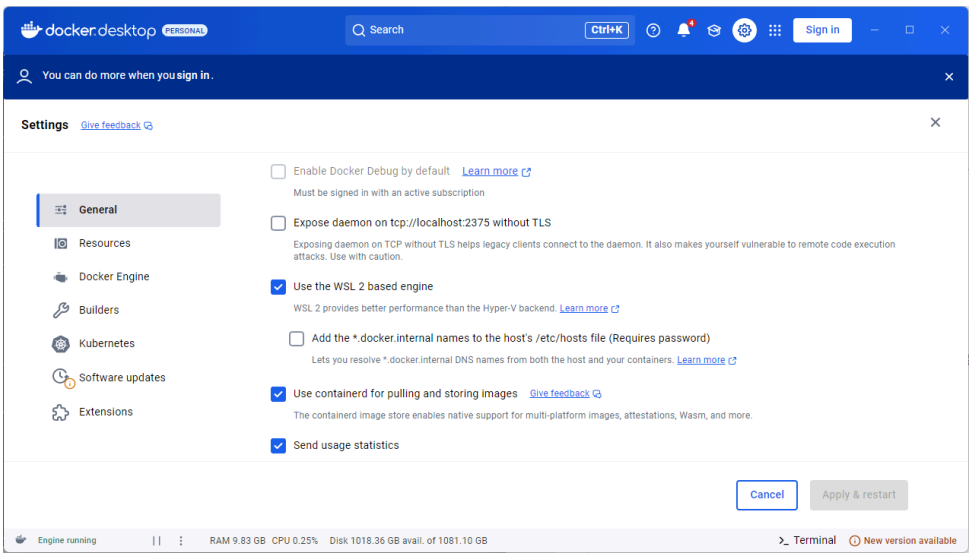
The command 'docker' could not be found in this WSL 2 distro.
We recommend to activate the WSL integration in Docker Desktop settings.

For details about using Docker Desktop with WSL 2, visit:

https://docs.docker.com/go/wsl2/
```

따라서 Elasticsearch는 WSL 환경에, 도커는 윈도우 환경에 설치하여 진행한다.

1.1. Docker Desktop의 WSL 2 백엔드 활성화 ——



1.2. Elasticsearch 설치 ——

<Run Elasticsearch locally>에서 제공하는 방법으로 ES를 설치한다: https://www.elastic.co/docs/solutions/search/run-elasticsearch-locally

- ```
  curl -fsSL https://elastic.co/start-local | sh
  ```
- `curl` 명령을 실행하면 도커로부터 필요한 의존성과 런타임 이미지를 다운로드하고 실행할 수 있다.

```
❯ curl -fsSL https://elastic.co/start-local | sh


  ______ _           _   _
 |  ____| |         | | (_)
 | |__  | | __ _ ___| |_ _  ___
 |  __| | |/ _` / __| __| |/ __|
 | |____| | (_| __ \ |_| | (__
 |______|_|__,_|___/__|_|___|
--------------------------------------------------
🚀 Run Elasticsearch and Kibana for local testing
--------------------------------------------------

ℹ  Do not use this script in a production environment

⏳ Setting up Elasticsearch and Kibana v9.0.0...

- Generated random passwords
- Created the elastic-start-local folder containing the files:
  - .env, with settings
  - docker-compose.yml, for Docker services
  - start/stop/uninstall commands
- Running docker compose up --wait

[+] Running 26/18
 ✔ kibana Pulled                                                          83.6s
 ✔ elasticsearch Pulled                                                   63.4s
 ✔ kibana_settings Pulled                                                 64.1s


[+] Running 6/6
 ✔ Network elastic-start-local_default             Created                 0.1s
 ✔ Volume "elastic-start-local_dev-kibana"         Created                 0.0s
 ✔ Volume "elastic-start-local_dev-elasticsearch"  Created                 0.0s
 ✔ Container es-local-dev                          Healthy                36.4s
 ✔ Container kibana_settings                       Exited                 35.9s
 ✔ Container kibana-local-dev                      Healthy                68.3s

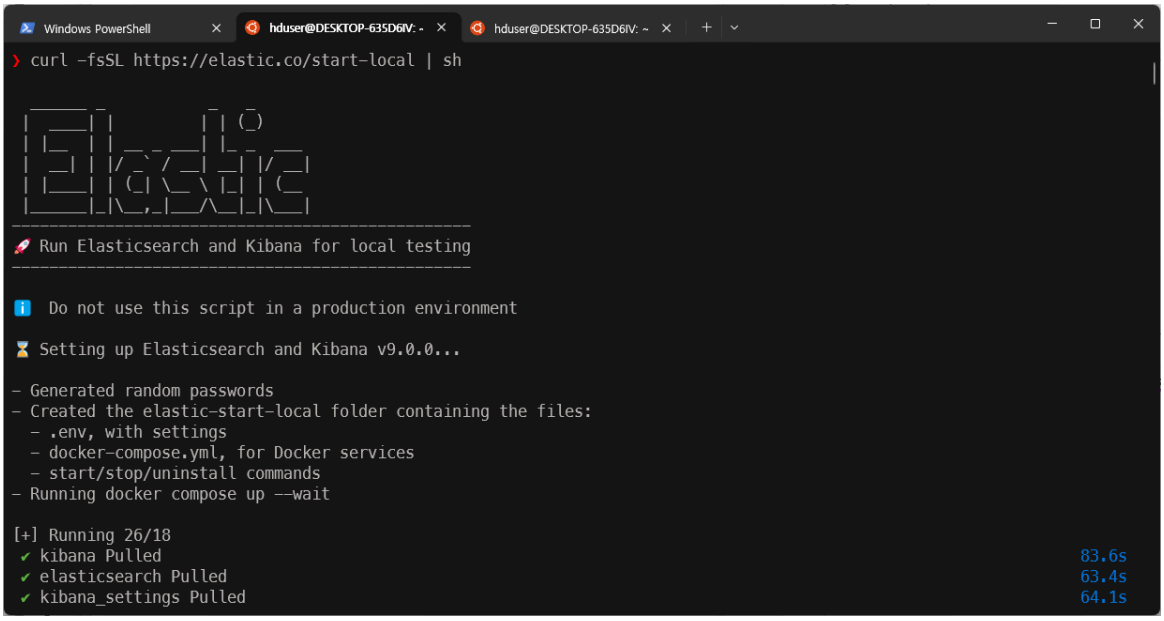
🎉 Congrats, Elasticsearch and Kibana are installed and running in Docker!

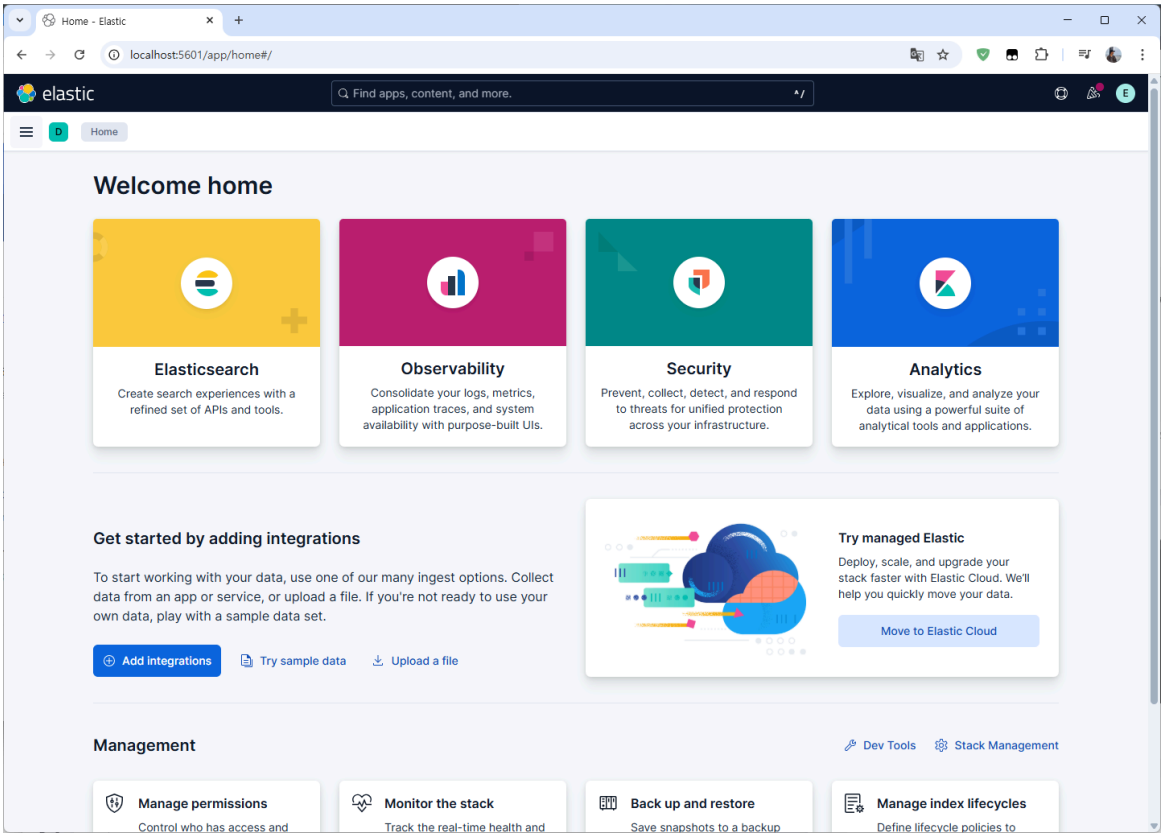
🌐 Open your browser at http://localhost:5601

   Username: elastic
   Password:

🔌 Elasticsearch API endpoint: http://localhost:9200
🔑 API key:


Learn more at https://github.com/elastic/start-local
```
```

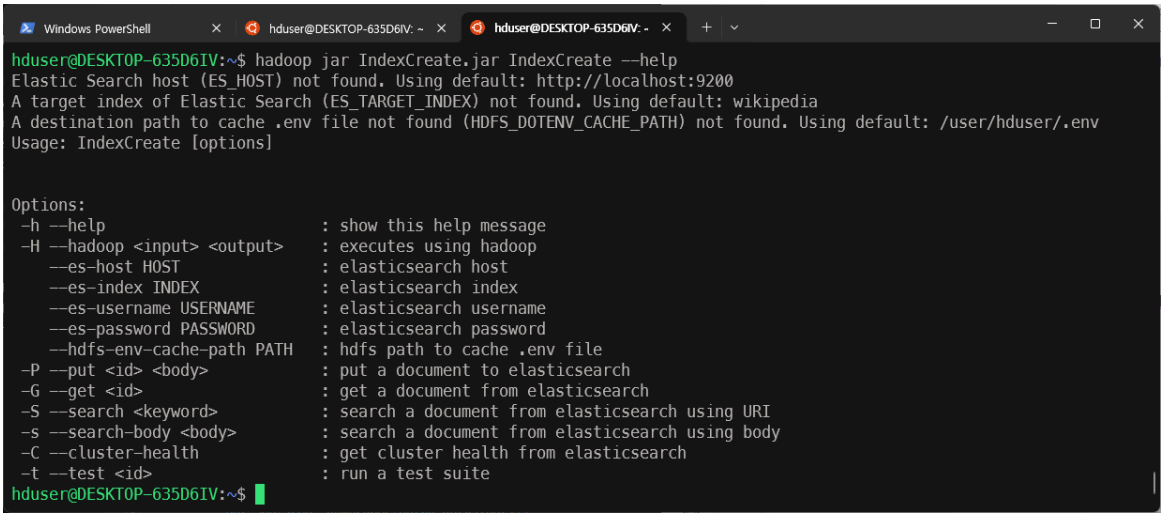




설치가 완료되고 서버가 동작하면 터미널 출력의 URL(http://localhost:5601)을 통해 elastic 대시보드에 접속할 수 있다.

2. Hadoop + Elasticsearch 연동 프로그램

본 과제를 달성하기 위해 프로그램을 작성하면서 디버깅 소요가 있었다. 이에 대응하여 하둡을 사용하지 않고 Elasticsearch에 작업을 요청하는 기능을 추가하고, 프로그램의 하둡 사용을 끄고 켤수 있도록 옵션화하였다.



2.1. Hadoop + Elasticsearch 연동 프로그램의 인덱스 생성 결과 ——

연동 프로그램은 `10K.ID.CONTENTS` 파일의 콘텐츠 부분을 `{"body": (콘텐츠)}` 형식으로 래핑하여 Elasticsearch에 인덱스하도록 하였다.

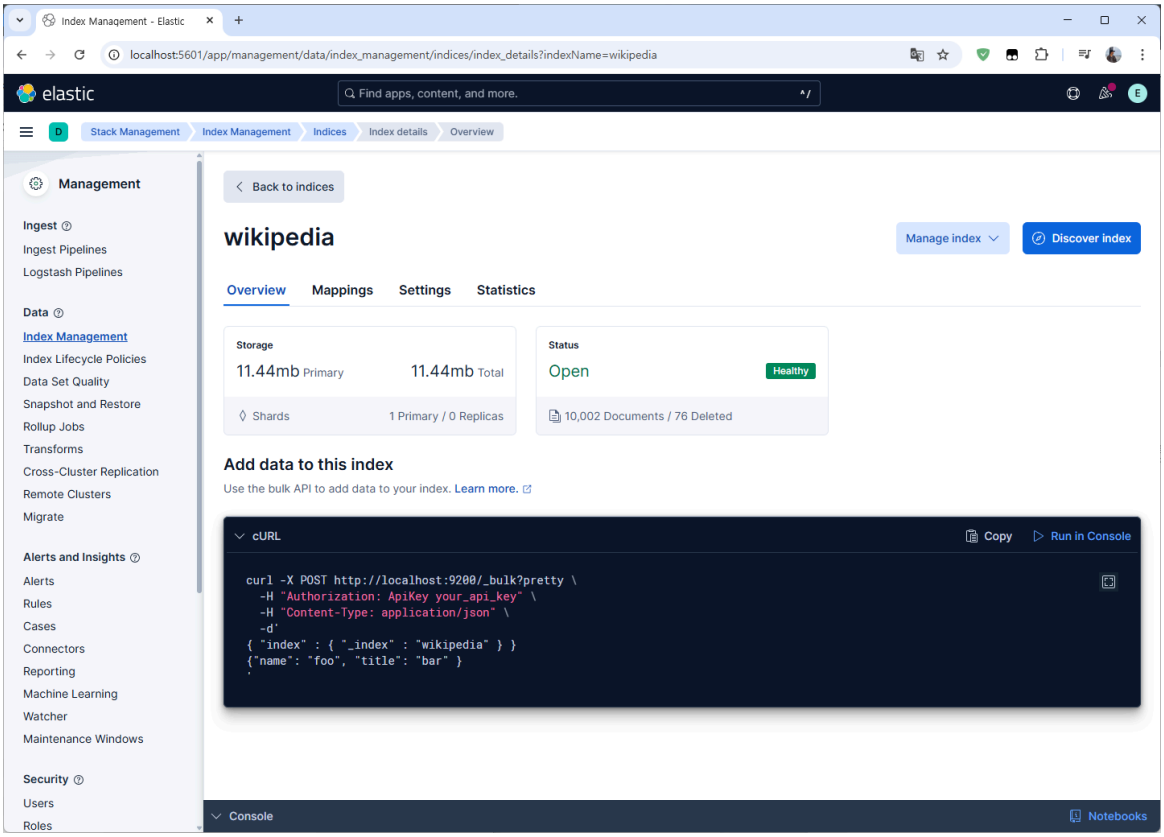
```
hduser@DESKTOP-635D6IV:~$ hadoop jar IndexCreate.jar IndexCreate --hadoop /user/
hduser/10K.ID.CONTENTS /tmp/jobOutput

2025-05-05 23:06:56,640 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to
ResourceManager at /0.0.0.0:8032
2025-05-05 23:06:57,047 WARN mapreduce.JobResourceUploader: Hadoop command-line option
parsing not performed. Implement the Tool interface and execute your application with
ToolRunner to remedy this.
2025-05-05 23:06:57,075 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding
for path: /tmp/hadoop-yarn/staging/hduser/.staging/job_1746289569867_0036
2025-05-05 23:06:57,882 INFO input.FileInputFormat: Total input files to process : 1
2025-05-05 23:06:57,960 INFO mapreduce.JobSubmitter: number of splits:1
2025-05-05 23:06:58,091 INFO mapreduce.JobSubmitter: Submitting tokens for job:
job_1746289569867_0036
2025-05-05 23:06:58,091 INFO mapreduce.JobSubmitter: Executing with tokens: []
2025-05-05 23:06:58,305 INFO conf.Configuration: resource-types.xml not found
2025-05-05 23:06:58,305 INFO resource.ResourceUtils: Unable to find 'resource-
types.xml'.
2025-05-05 23:06:58,472 INFO impl.YarnClientImpl: Submitted application
application_1746289569867_0036
2025-05-05 23:06:58,532 INFO mapreduce.Job: The url to track the job: http://DESKTOP-
635D6IV.:50000/proxy/application_1746289569867_0036/
2025-05-05 23:06:58,533 INFO mapreduce.Job: Running job: job_1746289569867_0036
2025-05-05 23:07:05,650 INFO mapreduce.Job: Job job_1746289569867_0036 running in uber
mode : false
2025-05-05 23:07:05,651 INFO mapreduce.Job:  map 0% reduce 0%

...(중략)...

        Stats
```

```
                Success=10000
        File Input Format Counters
                Bytes Read=9735683
        File Output Format Counters
                Bytes Written=9735683
Job completed successfully.
```

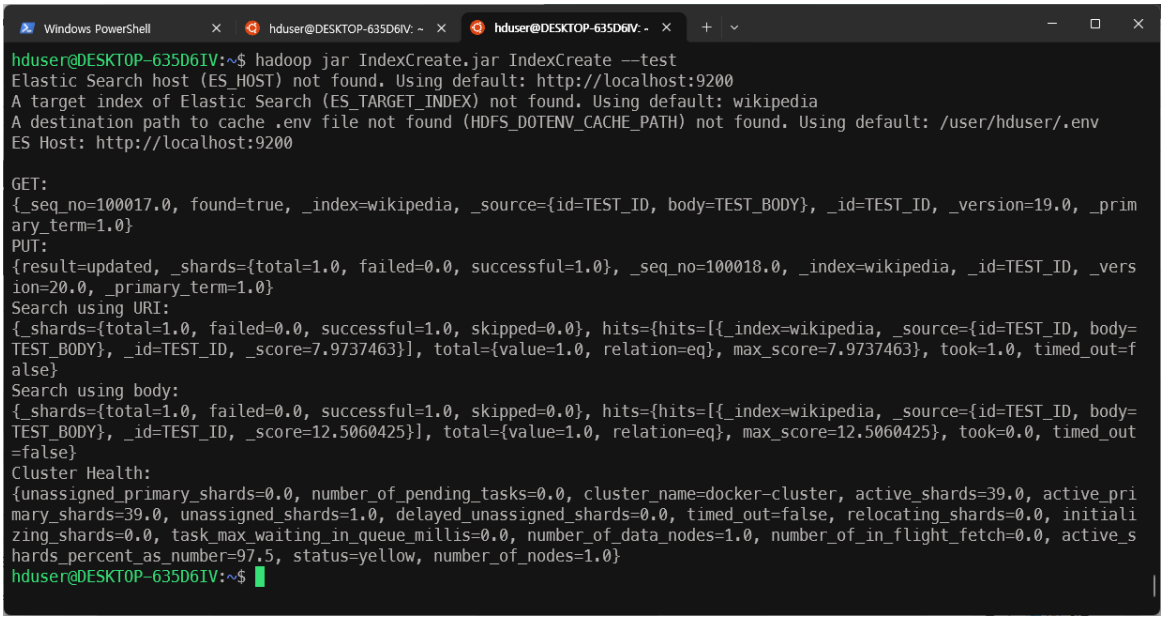


2.2. 검색 기능 검증 ——

```
hduser@DESKTOP-635D6IV:~$ hadoop jar IndexCreate.jar IndexCreate --test
Elastic Search host (ES_HOST) not found. Using default: http://localhost:9200
A target index of Elastic Search (ES_TARGET_INDEX) not found. Using default: wikipedia
A destination path to cache .env file not found (HDFS_DOTENV_CACHE_PATH) not found.
Using default: /user/hduser/.env
ES Host: http://localhost:9200

GET:
{_seq_no=100017.0, found=true, _index=wikipedia, _source={id=TEST_ID, body=TEST_BODY},
_id=TEST_ID, _version=19.0, _primary_term=1.0}
PUT:
{result=updated, _shards={total=1.0, failed=0.0, successful=1.0}, _seq_no=100018.0,
_index=wikipedia, _id=TEST_ID, _version=20.0, _primary_term=1.0}
Search using URI:
{_shards={total=1.0, failed=0.0, successful=1.0, skipped=0.0},
hits={hits=[{_index=wikipedia, _source={id=TEST_ID, body=TEST_BODY}, _id=TEST_ID,
_score=7.9737463}], total={value=1.0, relation=eq}, max_score=7.9737463}, took=1.0,
timed_out=false}
Search using body:
{_shards={total=1.0, failed=0.0, successful=1.0, skipped=0.0},
hits={hits=[{_index=wikipedia, _source={id=TEST_ID, body=TEST_BODY}, _id=TEST_ID,
_score=12.5060425}], total={value=1.0, relation=eq}, max_score=12.5060425}, took=0.0,
timed_out=false}
```

```
Cluster Health:
{unassigned_primary_shards=0.0, number_of_pending_tasks=0.0, cluster_name=docker-
cluster, active_shards=39.0, active_primary_shards=39.0, unassigned_shards=1.0,
delayed_unassigned_shards=0.0, timed_out=false, relocating_shards=0.0,
initializing_shards=0.0, task_max_waiting_in_queue_millis=0.0,
number_of_data_nodes=1.0, number_of_in_flight_fetch=0.0,
active_shards_percent_as_number=97.5, status=yellow, number_of_nodes=1.0}
```



1. title 검색

```
GET:
{_seq_no=100017.0, found=true, _index=wikipedia, _source={id=TEST_ID,
body=TEST_BODY}, _id=TEST_ID, _version=19.0, _primary_term=1.0}
```

콘텐츠 본문에서 일부를 발췌하여 별도의 타이틀을 생성하여 딕셔너리로 `{"title": (타이틀), "body": (콘텐츠)}`과 같이 구조화한 것이 아니므로, `id`를 타이틀로 간주하고 다음과 같이 쿼리하도록 하였다. 위 실행 결과에서 `id`는 `TEST_ID`로 되어있으나, 실제로는 `2978668`와 같이 `10K.ID.CONTENTS` 상에서 `\t`로 구분되어 주어지는 각 행의 첫째 값을 쿼리하도록 의도되었다.

2. body 검색

```
Search using body:
{_shards={total=1.0, failed=0.0, successful=1.0, skipped=0.0},
hits={hits=[{_index=wikipedia, _source={id=TEST_ID, body=TEST_BODY}, _id=TEST_ID,
_score=12.5060425}], total={value=1.0, relation=eq}, max_score=12.5060425},
took=0.0, timed_out=false}
```

3. url 검색

```
Search using URI:
{_shards={total=1.0, failed=0.0, successful=1.0, skipped=0.0},
hits={hits=[{_index=wikipedia, _source={id=TEST_ID, body=TEST_BODY}, _id=TEST_ID,
_score=7.9737463}], total={value=1.0, relation=eq}, max_score=7.9737463}, took=1.0,
timed_out=false}
```

4. 상태확인

```
Cluster Health:
{unassigned_primary_shards=0.0, number_of_pending_tasks=0.0, cluster_name=docker-
cluster, active_shards=39.0, active_primary_shards=39.0, unassigned_shards=1.0,
delayed_unassigned_shards=0.0, timed_out=false, relocating_shards=0.0,
initializing_shards=0.0, task_max_waiting_in_queue_millis=0.0,
number_of_data_nodes=1.0, number_of_in_flight_fetch=0.0,
active_shards_percent_as_number=97.5, status=yellow, number_of_nodes=1.0}
```